

Monthly Care Plan

Personalised nutrition and physical-activity guidance,
doctor-reviewed medications and notes — one plan per pregnancy month.

DATE	6 October 2026
STATUS	Weekly design built and tested — clinical review pending
SCOPE	Backend and API only (frontend is separate)
FULL SPEC	docs/design/2026-10-05 and 2026-10-06 care-plan design docs

Clinical review still required

Category cut-offs, trimester cut-offs, safety rules and the activity baseline have not been reviewed by an obstetrician or dietitian. Not ready for real patients until they are.

What we are building

Today the risk engine tells us **how risky** a patient is. This feature answers the next question: **what should she do about it?** For every pregnant patient the system keeps a **Care Plan for each pregnancy month**. The plan is one page that doctors and the patient can both open. Inside it, nutrition and exercise are written **once per pregnancy week**, with a short **progress summary** of how she has been doing, and a lighter layer of **quick advice** reacts to individual readings during the week.

Section	Who writes it	What the patient sees
Nutrition — meals, foods to eat and avoid, hydration, timing tips	Drafted by the AI (OpenRouter), checked by code. Doctors can add, edit and remove items.	Read-only
Physical activity — exercises with duration, frequency, intensity, what to avoid	Drafted by the AI, checked by code. Doctors can add, edit and remove items.	Read-only
Medication	Only doctors (providers) . The AI never writes or mentions medicines.	Read-only
Notes for the patient	Staff write them by hand.	Read-only — always visible

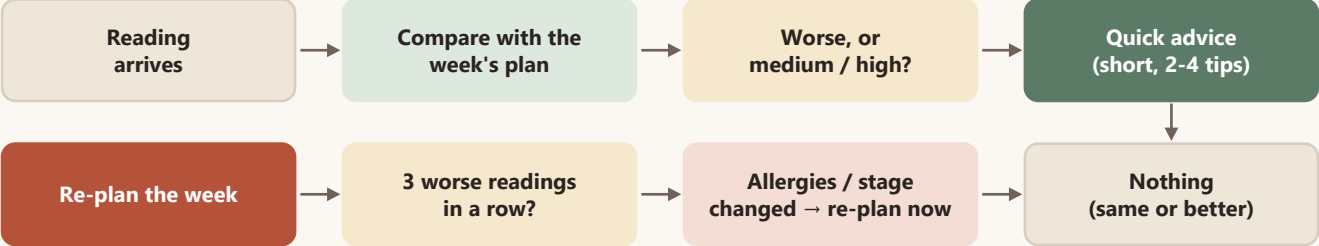
The plan page also shows the patient's **allergies and conditions**, read straight from her record (never copied), so everyone can see what the plan was built around.

The big picture

1. At 12 am local time on the first day of each pregnancy week — the weekly plan, written by the scheduled sweep (retried until it succeeds)



2. On each reading during the week — quick advice, and a re-plan only if a worse condition persists



A worse reading on its own never rewrites the week: it gets a short tip that looks at the plan she is already following. Only a worse condition that is still there hours later changes the plan.

What we decided

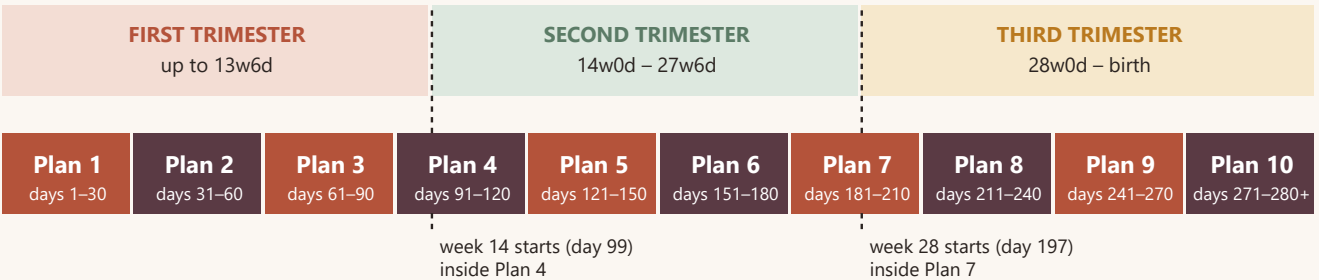
#	Decision
1	One plan per Pregnancy per pregnancy month . A month is 30 days from day 1 of the pregnancy . A patient onboarded at week 22 starts at Plan 6 — no empty earlier plans.
2	Day 1 and gestational age come from the EDD captured at onboarding (LMP or EDD is required). No EDD means no plan month — the API says so instead of guessing.
3	Trimesters: 1st up to 13w6d · 2nd 14w0d–27w6d · 3rd from 28w0d. Defined once in code.
4	Four sections for staff and patient: nutrition, exercise, medication, notes. Allergies and conditions are shown on the plan.
5	Nutrition and exercise come from an LLM via OpenRouter , once per pregnancy week , as two separate requests (one for each), each with its own sources and creation time. Code validates each answer before anyone sees it.
6	Medication is never AI-generated . No medicine, supplement, dose or calorie wording may appear in AI text.
7	Notes are always visible to the patient . The existing staff-only Clinical Notes are untouched.
8	Staff can create, edit and deactivate (never hard-delete) items. Patients cannot write anything.
9	Two new onboarding fields: food allergies and dietary preference . Editing them in the plan writes to the patient record.
10	Three statuses: in progress → reviewed → finalized , with reopen. Status never hides the plan from the patient.
11	A worse reading mid-week gets quick advice only ; the weekly plan changes mid-week only if the worse condition persists (3 consecutive worse readings; counted in readings only, no time rule) or her allergies, diet, conditions or stage change. Loosening waits for the next weekly plan. Enforced in code.
12	A doctor's edits stay on that patient's plan and survive every regeneration.
13	3 distinct staff members making the same correction (same hospital, region, trimester) → a suggestion for the hospital admin. Approved suggestions guide later prompts. Nothing crosses hospitals.
14	Frontend screens are out of scope. We build the API and the Postman docs.
15	Weeks are pregnancy weeks (week 17 = 17w0d–17w6d from day 1). A week lives in the monthly plan that holds its first day.
16	Each weekly plan starts with a progress summary computed by code : last week's readings as percentages by risk level, the main concerns, the trend against earlier readings this month. A week with no readings is planned from the whole month's readings.
17	Quick advice on a medium/high or worse-than-plan reading: 2–4 short tips, sources and a timestamp. At high risk, "contact your care team today" is set by code. It never replaces the weekly plan, and it stops showing once the week is re-planned.
18	Doctor review is weekly, and only medium/high weekly plans go to the review list.
19	A pregnancy week starts at 12 am where she lives . A scheduled job (every 15 minutes) writes the new week's plan without waiting for a reading, and creates it again if it failed or fell back to the generic plan.

Terminology. "Hospital" and "organization" mean the same thing in MomCare — one Organization row per hospital.

Months and trimesters

Doctors count pregnancy in **weeks**, from the first day of the last period, and divide it into **trimesters**. Our plan is a **30-day month** because it is easy for patients and staff, and matches the rhythm of antenatal visits. The two work together:

Concept	Role in the care plan
Month (30 days)	The container. One plan per month. It is the page staff review and sign, and it holds that month's medications and notes.
Trimester	Decides what the nutrition and exercise advice says, and is shown as a label on the plan ("Plan 5 · weeks 17–21 · second trimester").
Risk and vitals	Decide whether the plan stays the same or is updated during the month.



Day 1 = first day of the pregnancy (EDD – 280). Plans continue past Plan 10 if the pregnancy goes past term.

A trimester boundary can fall **inside** a plan. When it does, the advice switches at the boundary — the patient does not wait for the next month.

How the numbers are worked out

```
day_index      = days since (EDD - 280)          # None if there is no EDD
month_number   = day_index // 30 + 1
period_start   = (EDD - 280) + (month_number - 1) * 30
trimester      = from gestational weeks (cut-offs above)
```

A weekly plan is created by the **first reading of a new pregnancy week**, or by the scheduled sweep for a week nobody sent a reading in. A week belongs to the monthly plan that holds its **first day**, so a week that straddles a month edge is never split. A pregnancy with no readings at all gets no plan.

How the weekly plan and the quick advice are generated

Both start from a hook on each scored reading (the same pattern the AI Summary already uses) and run inside the reading's request. Failures are logged and **never fail or roll back the reading**; a scheduled sweep command starts the week for patients who sent no reading. The model is the platform-wide AI setting (the platform admin can switch it), so one switch changes summaries, plans and advice together.

The weekly plan

Step	What happens
1. Gather	Last week's readings, this month's readings, and her latest reading as her current state. A week with no readings uses the whole month; a month with none uses her last readings (the plan says so). No readings at all: no plan.
2. Progress summary — by code	Last week: how many readings, the share at low / medium / high risk, the most frequent abnormal vitals (e.g. blood pressure Stage 2 in 4 of 6 readings). This month: the trend against earlier readings (improving, steady or worse) and the change in low-risk readings in percentage points. Then the areas to work on this week. The text is assembled from these facts — the AI never writes a number .
3. Ask OpenRouter — twice	One request for nutrition and a separate one for exercise, each with its own prompt, word limit and sources, and each given the progress facts so the plan responds to them. Only categories and ages are sent — never names, IDs, CNIC or contact details.
4. Validate each answer	Schema check · sources present · banned-content check · word limit · allergy check · high-risk activity cap. One retry, then that section's fallback.
5. Fallback, per section	If OpenRouter fails, times out or stays invalid for a section, that section shows a safe generic baseline stored in code (naming MomCare's baseline as its source). The other section is unaffected. Never an error.
6. Store	A new, append-only version for the week, with the inputs it was built from. New content nobody has reviewed returns a reviewed or finalized monthly plan to in progress.

During the week: what each reading does

The reading compared with what the plan was written for	What happens
Allergies, diet, conditions or stage of pregnancy changed	The week's plan is rewritten at once — a newly recorded allergy cannot wait.
Worse (higher risk, or a vital category worse)	Quick advice if the condition is new; the weekly plan is not changed. If the worse condition is still there on the third reading in a row, the week is re-planned .
Medium or high risk, but not worse than the plan	Quick advice only if the condition differs from the plan's and from the last advice (identical readings cost nothing).
Same or better, low risk	Nothing. Loosening waits for the next weekly plan.

Worked example — a bad Monday

Time	Reading	What the patient sees
Monday 8 am	Low (first of the week)	The week's plan, written from last week's readings, with the progress summary

Time	Reading	What the patient sees
Monday noon	High	Quick advice: rest, sip water, breathing, contact your care team today. The weekly plan is unchanged.
Monday 4 pm	High again	Nothing new: the same condition (second worse reading in a row)
Monday 8 pm	Still high	The third worse reading in a row: the week is re-planned for her current condition and the quick advice is retired

Being too careful costs little; being too relaxed can cause harm. The two numbers in the persistence rule (3 readings in a row) is a constant in the code, ready for a clinician to adjust.

Safety guardrails (all enforced in code)

Guardrail	What it does
Structured output	The model must return JSON in a fixed format. Anything else is rejected.
Banned-content check	Rejects any text mentioning medication, tablets, capsules, pills, supplements, doses (mg / mcg / IU), calorie or weight targets. Also applied to doctors' edits and to correction text before it is saved.
Allergy check	Removes or rejects any food matching the patient's allergens. We never rely on the model alone.
High-risk activity cap	High risk gets only the gentle-baseline activity set, whatever the model wrote.
Quick advice and re-plan rules	Implemented in code, not left to the model: a worse reading gets quick advice only; the week is re-planned only if the worse condition persists, or her allergies, diet, conditions or stage change.
Quick-advice checks	Same banned-content and sources rules as a plan; short (at most 4 tips, about 100 words); tips naming an allergen she must not eat are dropped; at high risk strenuous tips are dropped and "contact your care team today" is always added by code.
Fallback plan	Safe generic baseline when OpenRouter is down or invalid. It is plant-based on purpose (it knows nothing about the patient, so a vegetarian or vegan is never told to eat meat, fish, egg or dairy); allergies are removed from it as from any plan.
Privacy	Only categories and ages are sent. No names, IDs, CNIC or contact details. The hospital's country is sent so foods fit where she lives (region alone produced pork and East-Asian dishes for a Pakistani hospital in a real test).
Short plans	Advice a nurse could hand over, not an essay: about 450 words in all (nutrition at most 300, exercise at most 120 as asked; hard limits of 380 and 160 in code). Short lists, no weekly rotation. An answer over the limits is rejected, retried once, then replaced by the baseline.
Sources always shown	Every nutrition and exercise section carries sources (1–3 authorities or guidance documents the model says the advice is based on) and a creation time. Links are dropped, and an answer with no usable source is rejected. The API marks them sources_verified: false : they are what the model says it relied on, from its training — nothing checks them against the documents, and a model can name a source wrongly.
Recognised sources only	The model is told to use only guidance from recognised authorities: the patient's national or regional health authority first (and the WHO regional office), then WHO, FIGO, ACOG, RCOG and NICE; never social media, blogs, forums or commercial sites; and to leave out anything it is not sure those authorities support. Honest limit: the model has no internet in this call and answers from its training, so this is an instruction, not something the system can check. A curated, clinician-approved reference pack of official recommendations would make it a guarantee — not built yet.
Disclaimer + warning signs	Every plan version carries: this is guidance, follow your provider's advice; severe headache, bleeding, reduced fetal movement, blurred vision or chest pain → seek care.
Model fixed, not learning	OpenRouter models do not learn from our calls. "Learning" is done by stored preferences added to prompts (Part 6).

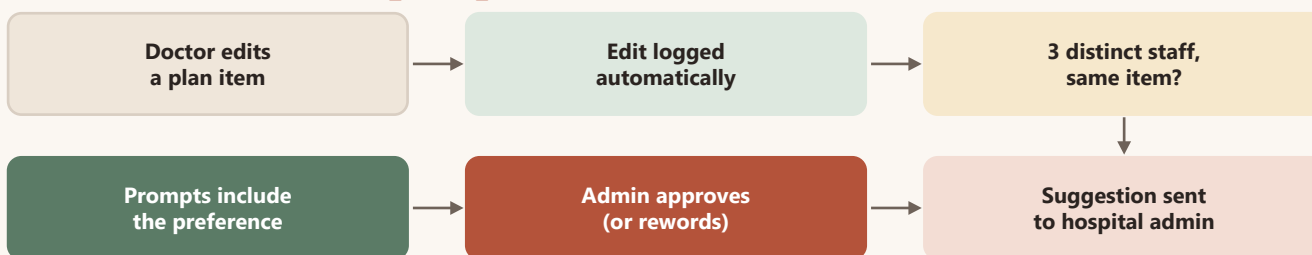
Why not a hand-written catalog? A catalog of foods and exercises for every region would need data from all over the world and a very long time to build. The LLM covers regions broadly; the guardrails above and doctor review keep it safe. **Trade-off:** LLM output has no pre-approved content, so clinical review of the rules and of sample plans is essential.

Doctor edits and learning

What happens when a doctor edits

- Edits are stored as an **adjustment layer** on top of the latest generated version. Regeneration never removes them.
- A removed item is never suggested again **for that patient**.
- **Next reading, condition unchanged:** no regeneration. The patient sees the plan with the doctor's edits, labelled "reviewed by your provider".
- **Condition changed:** the AI regenerates, and the patient's edits are sent as constraints ("this patient's doctor replaced oats with bread").
- Every edit is also logged automatically as JSON. No screen is involved.

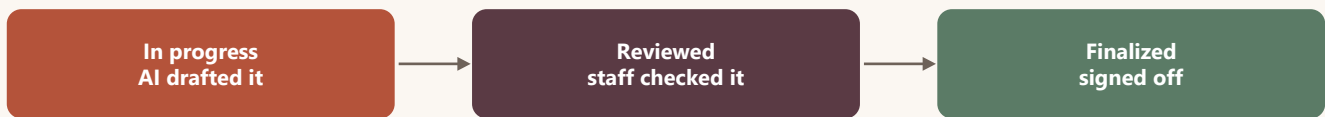
From one edit to a hospital preference



Question	Answer
What is counted?	The replaced or removed item (e.g. oats), not the replacement. Doctors may disagree on the replacement (paratha vs bread) but agree oats do not fit.
Counted how?	3 distinct staff members (any hospital staff role). One person repeating a habit does not count.
Counted where?	Within one hospital , for the same region and trimester . Never across hospitals.
"Oats" vs "oatmeal"?	The AI returns one standard English name per item; a small synonym list in code folds common variants; the admin can merge duplicates.
Admin approves	The text (optionally reworded) becomes a hospital preference. The top ~10 relevant ones are added to prompts. Admin can deactivate later.
Admin rejects	No hospital preference. Patients already edited keep their edits. Not re-suggested until 3 new distinct-doctor corrections arrive.
Safety	Correction text is passed to the model as data, not instructions and is checked by the banned-content filter before saving.

Level	Affects	Needs approval
Edit on a patient's plan	That patient only	No — automatic
Hospital preference	All patients in that hospital	Yes — hospital admin

Statuses and permissions



Reopen moves a plan back to in progress. Every transition records who and when. **Finalized** plans are locked until reopened. Editing a **reviewed** plan returns it to in progress. The patient always sees the current plan, even while it is in progress, labelled "suggested automatically" or "reviewed by your provider".

Who can do what

Action	Provider	Nurse / Care mgr	Hospital admin	Patient
Read the plan	Yes	Yes	Yes	Own plan, read-only
Edit nutrition / exercise items	Yes	Yes	Yes	No
Medications (create, edit, deactivate)	Yes	No	No	No
Notes	Yes	Yes	Yes	No
Allergies and conditions	Yes	Yes	Yes	No
Mark reviewed	Yes	Yes	Yes	No
Finalize / reopen	Yes	No	Yes	No
Approve or reject hospital preferences	No	No	Yes	No

- Everything is scoped to the user's own hospital. **Another hospital's plan returns 404, never 403.**
- Delete means deactivate:** the item disappears from normal lists and from the patient's view, and the record (who, when) is kept.
- The platform admin has no access — that app stays an empty placeholder.

Data model

New app: momcare_platform/modules/pregnancy/care_plans/ (app label care_plans). It sits under *modules* because it depends on readings and risk (dependency direction is modules → core). It reuses the existing OpenRouter client and shared helpers. Every table gets row-level security; every ID is a UUID.

Table	What it holds	Scoped by
CarePlan	pregnancy, month number, period start/end, status, reviewed and finalized by/at. One per pregnancy per month.	pregnancy → patient → location → organization
CarePlanSectionVersion	Each generated nutrition or exercise version: content JSON, the inputs it was built from (JSON), state key, triggering reading, model name, fallback flag. Append-only.	via care plan
CarePlanAdjustment	Staff additions, edits and removals on nutrition/exercise (JSON). Deactivatable.	via care plan
CarePlanMedication	Medication text, author. Deactivatable. Provider-only.	via care plan
CarePlanNote	Note text, author. Deactivatable. Always patient-visible.	via care plan
PlanCorrection	Append-only JSON log of every doctor edit: original, replacement, region, trimester, section, item key, doctor.	organization
HospitalPreference	Suggested / approved / rejected / inactive preferences with guidance JSON, supporting-doctor count, who decided.	organization
Patient (existing)	New fields: food allergies (list), dietary preference (none / vegetarian / vegan / other).	existing

Region is derived from the patient's country (never stored on the plan). Gestational age is always derived from the EDD, never stored. Conditions come from the existing yes / no / unknown answers on the pregnancy (chronic hypertension, diabetes, previous gestational diabetes, previous pre-eclampsia, and others).

API endpoints

All routes are under `/api/` and mounted explicitly in `config/api_router.py`. **Reads are one aggregate call; writes are per section** because permissions and validation differ (medication is provider-only; nutrition and exercise edits also feed the corrections log).

Endpoint	Purpose	Who
GET <code>/care-plans/?pregnancy=</code>	List the plans of a pregnancy.	Staff
GET <code>/care-plans/{id}/</code>	One plan, everything in one response : month, weeks, trimester, status, nutrition, exercise, medications, notes, allergies and conditions.	Staff
GET <code>/pregnancies/{id}/current-care-plan/</code>	The current plan (the screen's entry point).	Staff; patient (own, read-only)
GET <code>/pregnancies/{id}/current-care-plan/nutrition/ · /exercise/</code>	The current plan's nutrition or exercise on its own : content with staff edits applied, sources , <code>sources_verified</code> , <code>generated_at</code> , label and status. An older month is read through GET <code>/care-plans/{id}/</code> , whose nutrition and exercise blocks are already separate.	Staff; patient (own, read-only)
POST <code>/care-plans/{id}/adjustments/</code>	Add a nutrition or exercise item.	Staff
PATCH <code>/care-plans/{id}/adjustments/{adj_id}/</code>	Edit an item.	Staff
DELETE <code>/care-plans/{id}/adjustments/{adj_id}/</code>	Remove an item (deactivates).	Staff
POST · PATCH · DELETE <code>/care-plans/{id}/medications/...</code>	Create, edit, deactivate a medication.	Provider only
POST · PATCH · DELETE <code>/care-plans/{id}/notes/...</code>	Create, edit, deactivate a note for the patient.	Staff
PATCH <code>/care-plans/{id}/allergies-conditions/</code>	Edit allergies, dietary preference and conditions. Writes through to the patient and pregnancy, then regenerates the plan.	Staff
POST <code>/care-plans/{id}/review/</code>	Mark reviewed.	Staff
POST <code>/care-plans/{id}/finalize/</code>	Sign off.	Provider, hospital admin
POST <code>/care-plans/{id}/reopen/</code>	Back to in progress.	Provider, hospital admin
GET <code>/care-plan-preferences/?status=</code>	Suggestions and approved preferences.	Hospital admin
POST <code>/care-plan-preferences/{id}/approve/</code>	Approve a suggestion.	Hospital admin
POST <code>/care-plan-preferences/{id}/reject/</code>	Reject a suggestion.	Hospital admin
POST <code>/care-plan-preferences/{id}/deactivate/</code>	Stop using an approved preference.	Hospital admin
PATCH <code>/care-plan-preferences/{id}/</code>	Reword a suggestion before approving.	Hospital admin
GET <code>/patients/?workflow=care_plan_review</code>	Existing patient list: this month's plan still in progress (a query param, not a new queue endpoint).	Staff

Endpoint	Purpose	Who
GET /patients/?workflow=care_plan_missing	Existing patient list: no plan this month (no readings yet, or no EDD).	Staff
GET /patients/dashboard-kpis/ (existing)	Gains workflow.care_plan_review and workflow.care_plan_missing counts (same conditions as the list filters, so they always agree).	Staff

Lists use the standard pagination envelope (count, page, page_size, total_pages, next, previous, results). The Postman collection is updated using plain CRUD names (Create / Get / List / Update / Delete).

What the AI must return — two separate answers

Nutrition and exercise are two separate requests, so the model returns two separate answers, each ending with its own sources. Each food or exercise carries a standard English `item_key`, which is how repeated corrections are matched (oats and oatmeal both become oats). Every stored version also records when it was created.

Nutrition answer

```
{
  "meals": [{ "slot": "breakfast", "item_key": "paratha", "text": "Paratha with dahi" }],
  "foods_to_eat": [{ "item_key": "spinach", "text": "..."}],
  "foods_to_avoid": [{ "item_key": "raw_egg", "text": "..."}],
  "hydration": "...",
  "timing_tips": [...],
  "sources": ["WHO recommendations on antenatal care"]
}
```

Exercise answer

```
{
  "activities": [{ "item_key": "walking", "text": "...",
    "duration_minutes": 20, "frequency_per_week": 5, "intensity": "light" }],
  "avoid": [{ "item_key": "heavy_lifting", "text": "..."}],
  "stop_signs": [...],
  "sources": ["WHO guidelines on physical activity"]
}
```

Quick advice answer

```
{
  "tips": ["Rest calmly and lie on your left side.", "Sip small amounts of plain water."],
  "contact_care_team": true,
  "sources": ["WHO recommendations on antenatal care"]
}
```

Progress summary (computed by code, stored with the week)

```
{
  "facts": { "week_number": 21, "based_on": "last_week",
    "last_week": { "readings": 6, "risk_percent": { "low": 33, "medium": 50, "high": 17 },
      "concerns": [{ "axis": "bp", "category": "Stage 2", "count": 4, "of": 6 } ] },
    "trend": { "direction": "improved", "earlier_low_percent": 20, "recent_low_percent": 33,
      "low_risk_points_change": 13 },
    "focus": [ "bp", "hemoglobin" ] },
  "text": "Last week (week 20) you had 6 readings: 33% low risk, 50% medium and 17% high. ..."
}
```

A stored correction looks like this

```
{
  "organization": "<hospital>", "region": "South Asia", "trimester": 2,
  "section": "nutrition", "item_key": "oats", "action": "replace",
  "original": { "slot": "breakfast", "text": "Oatmeal with milk" },
  "replacement": { "slot": "breakfast", "text": "Paratha with dahi" },
  "edited_by": "<doctor>", "created_at": "2026-10-06T09:12:00Z"
}
```

An approved hospital preference stores its guidance the same way (JSON payload with the item key, the guidance text and the replacements doctors used), so it can be added to a prompt as plain data.

Testing, status and scope

How we will test it

- API-level integration tests on real Postgres; only the OpenRouter call is replaced.
- Pure-function tests for the month and trimester maths, the state key, the tighten / loosen rule, the banned-content and allergy checks, and item-key matching.
- Security tests proven by **fault injection** (the test must fail without the protection): cross-hospital 404 on every endpoint; patients cannot write; nurses cannot write medication; no medicine or supplement text can be stored or returned; edits survive regeneration; a removed item is not re-suggested; preferences never leak across hospitals.
- Row-level security is checked with the existing verification script, and the import-boundary check is part of standard verification.

Build status

Step	What	Status
1	Models, migrations, row-level security, month / trimester maths, statuses, read endpoints.	Done
2	Generation: state and state key, tighten / loosen, OpenRouter call, validation, fallback, versions.	Done
3	Staff editing: adjustments, medications, notes, allergies and conditions write-through, onboarding fields.	Done
4	Corrections log, suggestions, hospital preferences and admin endpoints.	Done
5	Postman collection (Care Plans and Care Plan Preferences folders) and documentation.	Done
6	Weekly plans, code-computed progress summary, quick reading advice, persistence rule, week-start sweep.	Done
—	Frontend screens, scheduling the sweep command, clinical review of rules and content.	Not done

Not in scope

- Frontend screens (Care Plan tab, patient mobile page) — separate project.
- Billing; Neuro_RPM's customizable care-plan form designer; a medication catalog.
- Any AI-written medication or supplement text.
- Email (in-portal only), cross-hospital learning, model fine-tuning.
- Languages other than English; calorie or weight targets.

Open items and the clinical caveat

Item	Status
OpenRouter model	Done: uses the existing platform-wide AI model setting.
Which reading counts as latest	Done: the single latest reading; a vital missing from it keeps its previous category; never recorded stays unknown.
Generation trigger	Done: hook on every scored reading + sweep command (manage.py sweep_care_plans) as a safety net. Schedule the sweep next to escalate_alerts.
Demo data	Asia.
Token limit and timeout	Care plans use their own (1500 tokens, 20 seconds); the summary word cap does not apply.
Persistence numbers	3 consecutive worse readings re-plan the week (counted in readings only). A constant in the code, awaiting clinical review.
Small model risk	A small model follows strict JSON and safety rules less reliably; code checks catch failures and the platform admin can switch models without code changes.
Verified	Automated tests, import-boundary check, database row-level security probe on all 7 tables, and fault injection of the main protections (see the build report).

Clinical review is required before real patients use this. The vital category cut-offs, the trimester cut-offs, the safety rules and the high-risk activity baseline have **not** been reviewed by an obstetrician or dietitian, and LLM output has no pre-approved content. The feature will be labelled "unreviewed" in code and documentation, and must not be described as ready for real patients until a clinician has reviewed the rules and sample plans.

Full written specification: docs/design/2026-10-05-care-plan-design.md in the backend repository.